Course: Cryptography and Network Security
Code: CSN15403
Branch: B.Tech -5th Semester

Lecture – 15 : Message Authentication and Digital Signatures

Faculty & Coordinator : Dr. J Sathish Kumar (JSK)

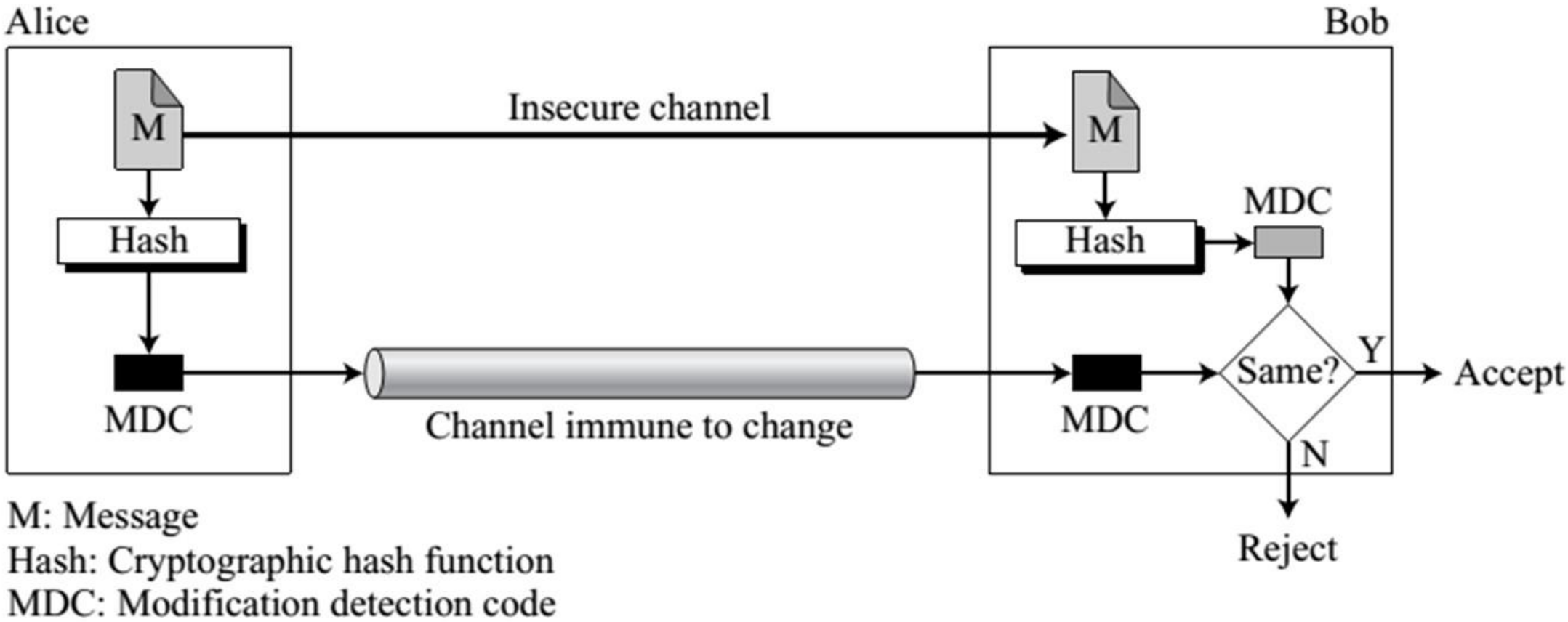
Department of Computer Science and Engineering

Motilal Nehru National Institute of Technology Allahabad,
Prayagraj-211004

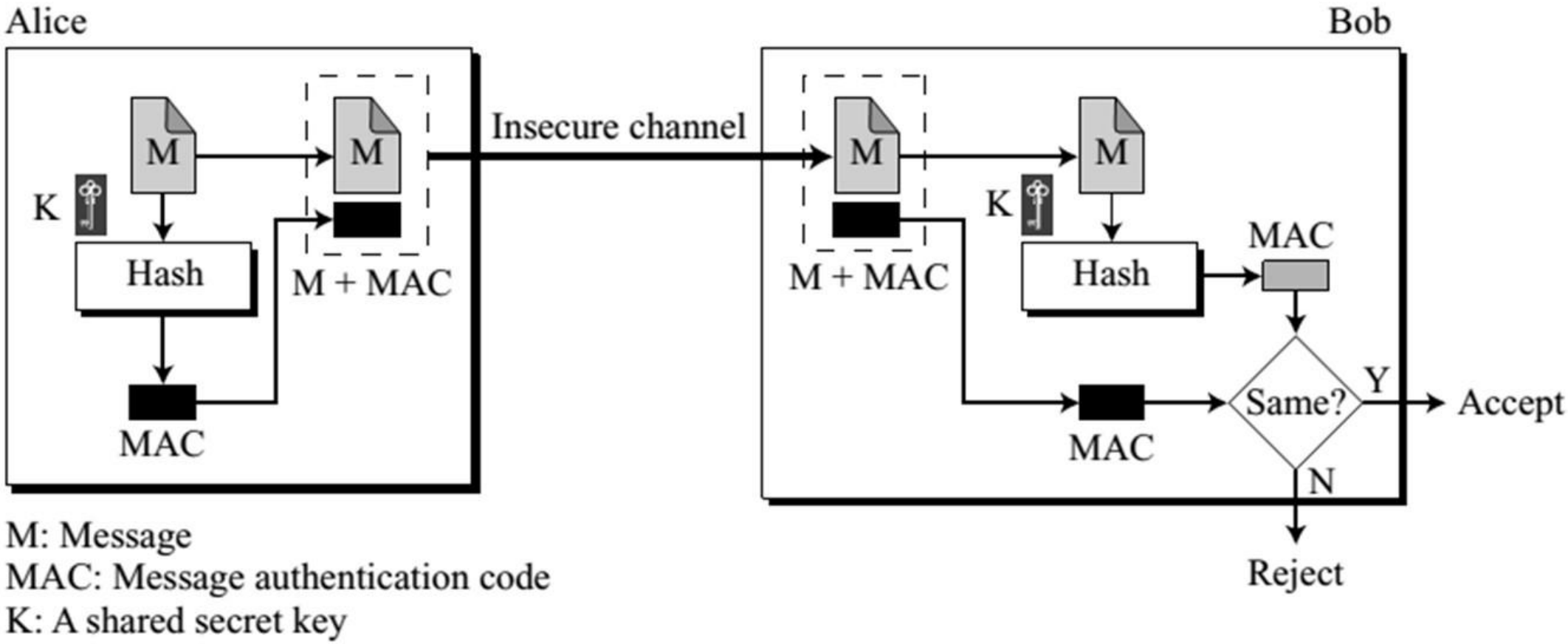
MESSAGE AUTHENTICATION

- A message digest guarantees the integrity of a message.
  - It guarantees that the message has not been changed.
- A message digest, however, does not authenticate the sender of the message.
  - When Alice sends a message to Bob, Bob needs to know if the message is coming from Alice.
- To provide message authentication, Alice needs to provide proof that it is Alice sending the message and not an impostor.
- A message digest per se cannot provide such a proof.
- The digest created by a cryptographic hash function is normally called a modification detection code (MDC).
  - The code can detect any modification in the message.
- What we need for message authentication (data origin authentication) is a message authentication code (MAC).

Modification detection code (MDC)



Message Authentication Code (MAC)



M: Message
MAC: Message authentication code
K: A shared secret key

Message Authentication Code (MAC)

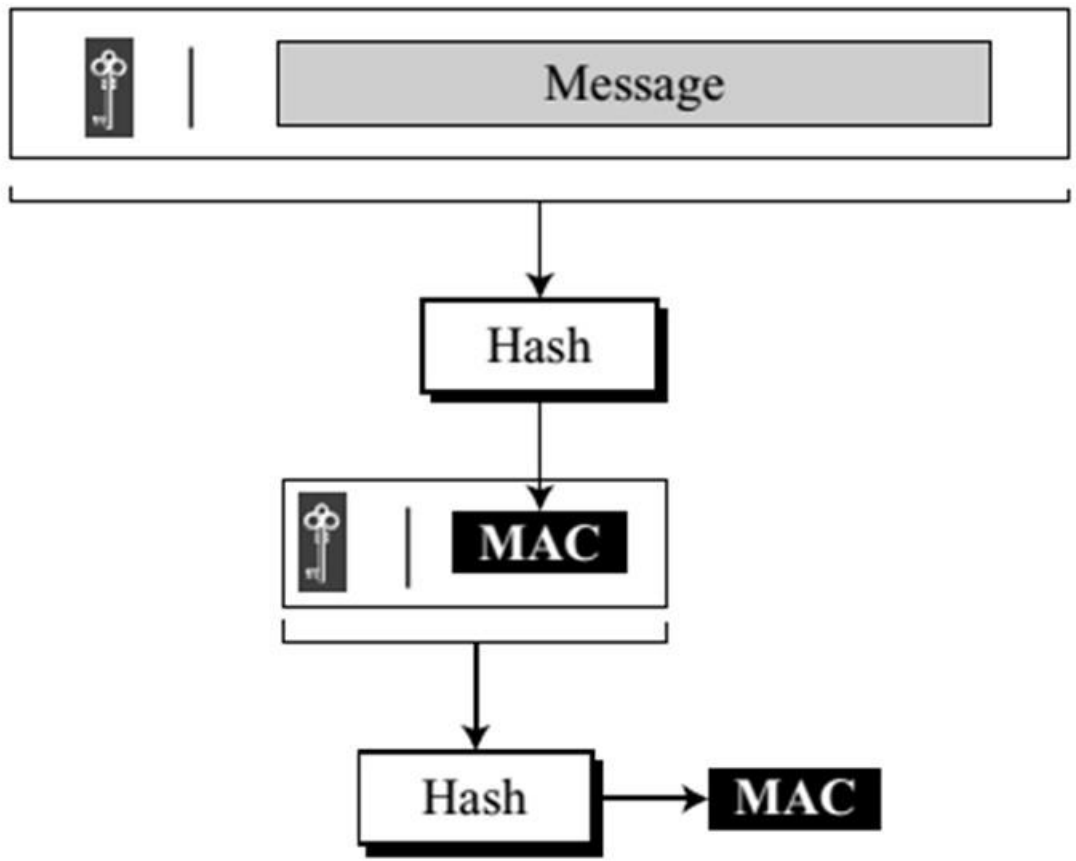
- Alice uses a hash function to create a MAC from the concatenation of the key and the message, h (K|M).
- She sends the message and the MAC to Bob over the insecure channel.
- Bob separates the message from the MAC.
- He then makes a new MAC from the concatenation of the message and the secret key.
- Bob then compares the newly created MAC with the one received.
- If the two MACs match, the message is authentic and has not been modified by an adversary.
- The MAC we have described is referred to as a prefix MAC because the secret key is appended to the beginning of the message.
- We can have a postfix MAC, in which the key is appended to the end of the message.
- We can combine the prefix and postfix MAC, with the same key or two different keys.

Security of a MAC

- Suppose Eve has intercepted the message M and the digest h(K| M). How can Eve forge a message without knowing the secret key? There are three possible cases:
- If the size of the key allows exhaustive search, Eve may prepend all possible keys at the beginning of the message and make a digest of the (K| M) to find the digest equal to the one intercepted. She then knows the key and can successfully replace the message with a forged message of her choosing.
- The size of the key is normally very large in a MAC, but Eve can use another tool: the preimage attack. She uses the algorithm until she finds X such that h(X) is equal to the MAC she has intercepted. She now can find the key and successfully replace the message with a forged one. Because the size of the key is normally very large for exhaustive search, Eve can only attack the MAC using the preimage algorithm.
- Given some pairs of messages and their MACs, Eve can manipulate them to come up with a new message and its MAC.

The security of a MAC depends on the security of the underlying hash algorithm

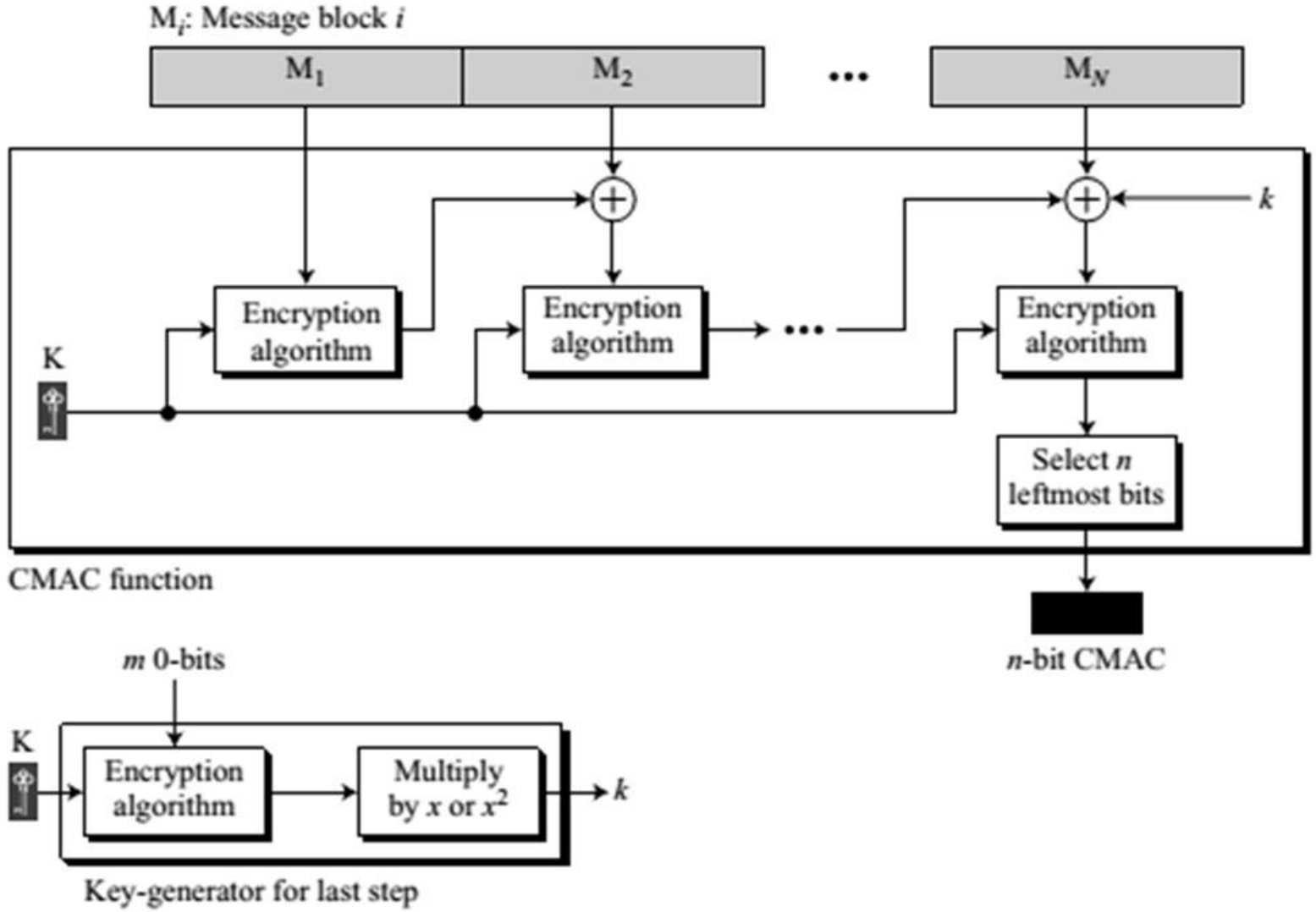
Nested MAC (OR) HMAC (Hashed MAC)



CMAC, or CBCMAC

CMAC, or CBCMAC is also called Data Authentication Algorithm.

The method is similar to the cipher block chaining (CBC) mode



Digital Signature

Digital Signature

- A person signs a document to show that it originated from her or was approved by her.
- The signature is proof to the recipient that the document comes from the correct entity.
- When Alice sends a message to Bob, Bob needs to check the authenticity of the sender; he needs to be sure that the message comes from Alice and not Eve.
- Bob can ask Alice to sign the message electronically.
- In other words, an electronic signature can prove the authenticity of Alice as the sender of the message.
- We refer to this type of signature as a digital signature.

Digital Signature: Inclusion

- A conventional signature is included in the document; it is part of the document.
- When we write a check, the signature is on the check; it is not a separate document.
- But when we sign a document digitally, we send the signature as a separate document.
- The sender sends two documents: the message and the signature.
- The recipient receives both documents and verifies that the signature belongs to the supposed sender.
- If this is proven, the message is kept; otherwise, it is rejected.

Digital Signature: Verification Method

- For a conventional signature, when the recipient receives a document, she compares the signature on the document with the signature on file.
- If they are the same, the document is authentic.
- The recipient needs to have a copy of this signature on file for comparison.
- For a digital signature, the recipient receives the message and the signature.
- A copy of the signature is not stored anywhere.
- The recipient needs to apply a verification technique to the combination of the message and the signature to verify the authenticity.

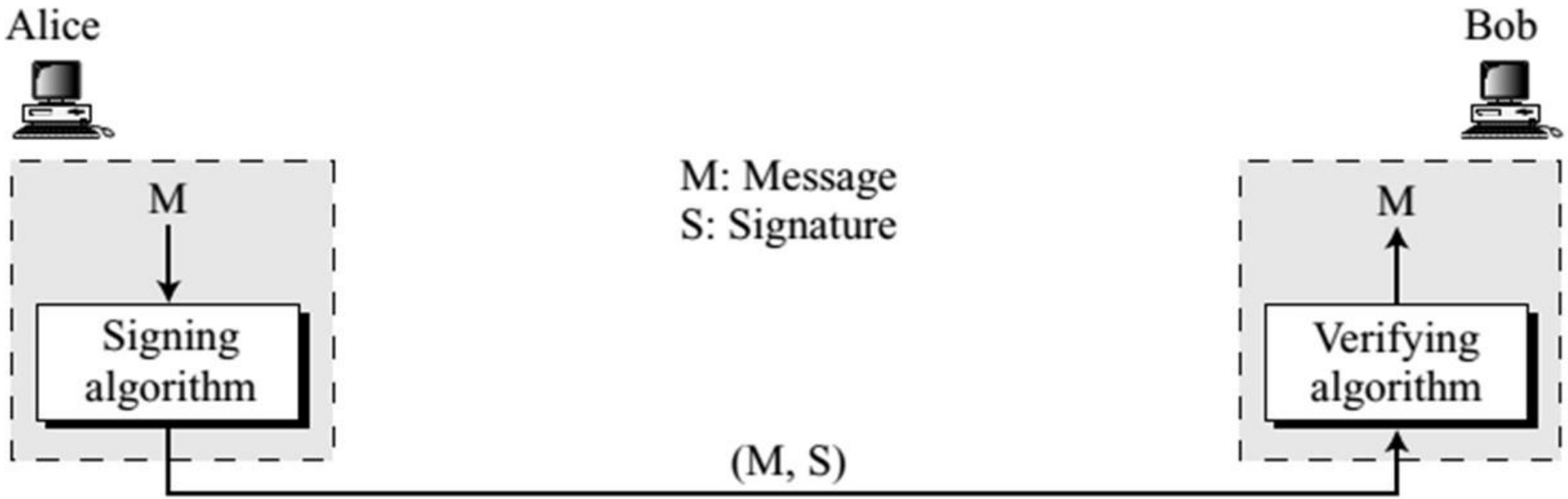
Digital Signature: Relationship

- For a conventional signature, there is normally a one-to-many relationship between a signature and documents.
- A person uses the same signature to sign many documents.
- For a digital signature, there is a one-to-one relationship between a signature and a message.
- Each message has its own signature. The signature of one message cannot be used in another message.
- If Bob receives two messages, one after another, from Alice, he cannot use the
signature of the first message to verify the second.
- Each message needs a new signature.

Digital Signature: Duplicity

- In conventional signature, a copy of the signed document can be distinguished from the original one on file.
- In digital signature, there is no such distinction unless there is a factor of time (such as a timestamp) on the document.
- For example,
  - Suppose Alice sends a document instructing Bob to pay Eve.
  - If Eve intercepts the document and the signature, she can replay it later to get money again from Bob.

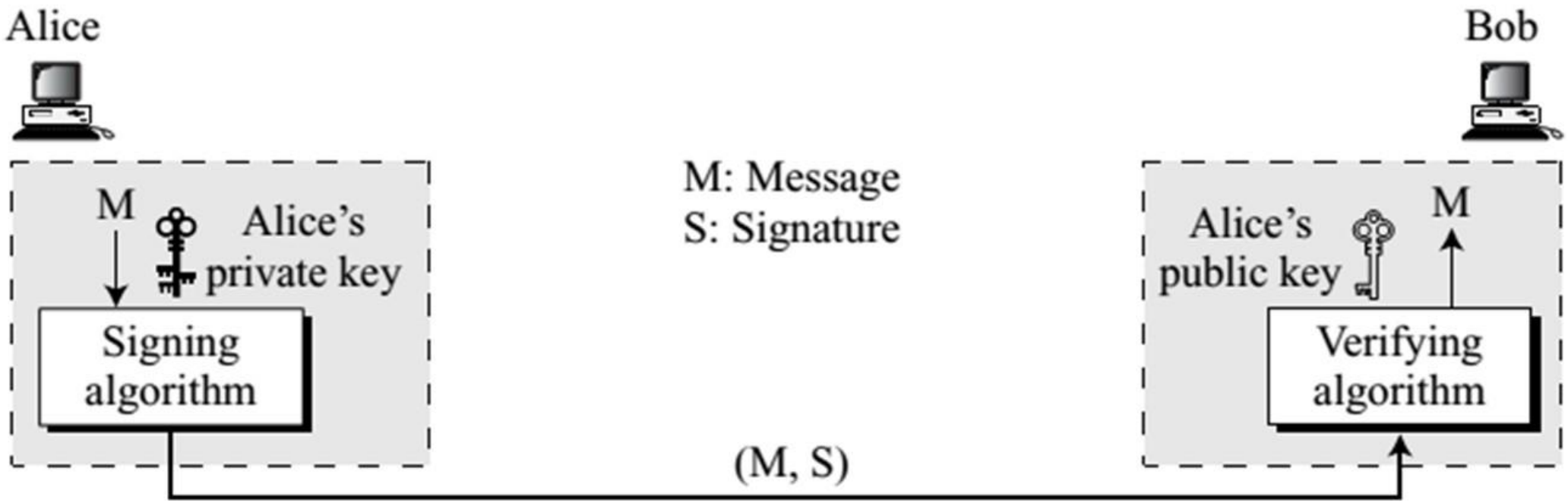
Digital Signature Process



Need for Keys

- A conventional signature is like a private "key" belonging to the signer of the document. The signer uses it to sign documents; no one else has this signature.
- The copy of the signature is on file like a public key; anyone can use it to verify a document, to compare it to the original signature.
- In a digital signature, the signer uses her private key, applied to a signing algorithm, to sign the document.
- The verifier, on the other hand, uses the public key of the signer, applied to the verifying algorithm, to verify the document.
- Note that when a document is signed, anyone, including Bob, can verify it because everyone has access to Alice's public key.
- Alice must not use her public key to sign the document because then anyone could forge her signature.

Adding key to the digital signature process

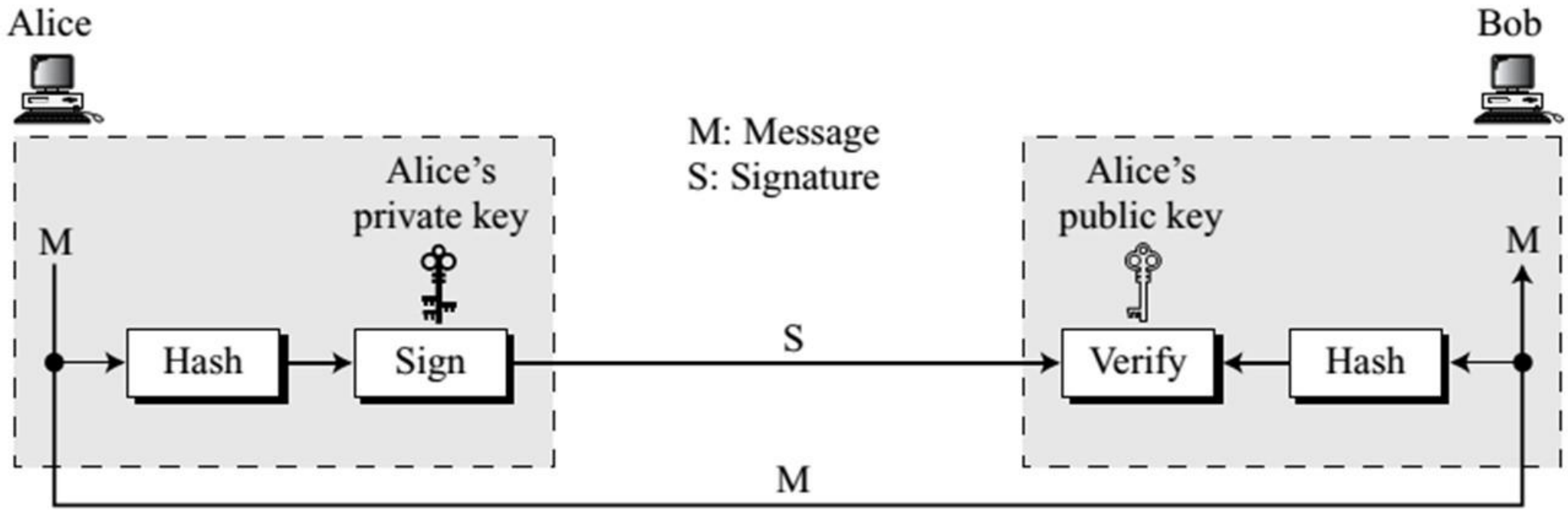


Digital Signature

- A digital signature needs a public-key system. The signer signs with her private key; the verifier verifies with the signer's public key.

- A cryptosystem uses the private and public keys of the receiver: a digital signature uses the private and public keys of the sender.

Signing the Digest



Digital Signature SERVICES

- Several security services were discussed including
  - message confidentiality,
  - message authentication,
  - message integrity, and
  - nonrepudiation.

- A digital signature can directly provide the last three;

- For message confidentiality we still need encryption/decryption.

Digital Signature : Message Authentication

- A secure digital signature scheme, like a secure conventional signature (one that cannot be easily copied) can provide message authentication (also referred to as data-origin authentication).
- Bob can verify that the message is sent by Alice because Alice's public key is used in verification.
- Alice's public key cannot verify the signature signed by Eve's private key.
- A digital signature provides message authentication

Digital Signature : Message Integrity

- The integrity of the message is preserved even if we sign the whole message because we cannot get the same signature if the message is changed.

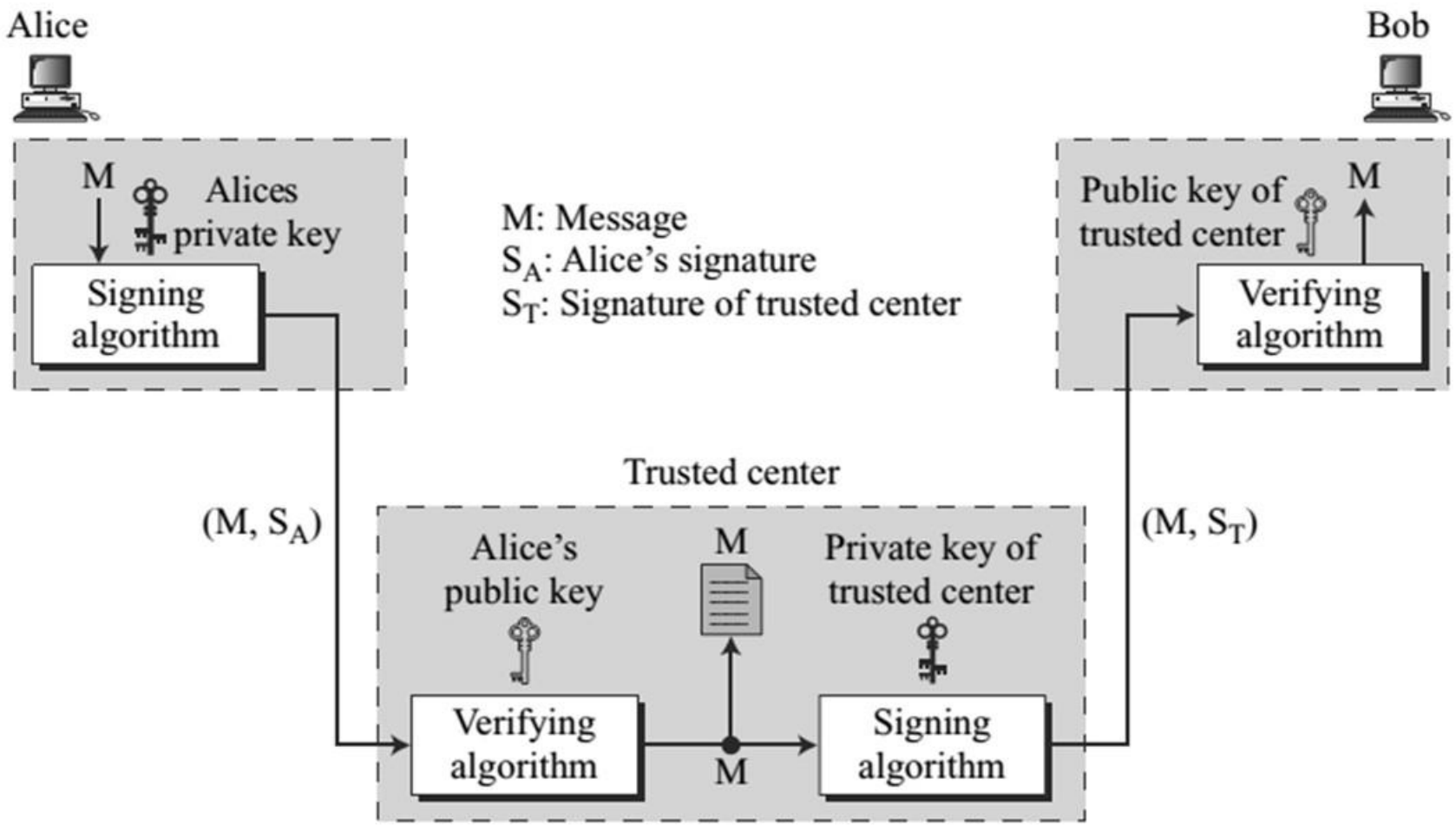
- The digital signature schemes today use a hash function in the signing and verifying algorithms that preserve the integrity of the message.

- A digital signature provides message integrity

Digital Signature : Nonrepudiation

- If Alice signs a message and then denies it, can Bob later prove that Alice actually signed it?
- For example, if Alice sends a message to a bank (Bob) and asks to transfer $10,000 from her account to Ted's account, can Alice later deny that she sent this message?
- With the scheme we have presented so far, Bob might have a problem.
- Bob must keep the signature on file and later use Alice's public key to create the original message to prove the message in the file and the newly created message are the same.
- This is not feasible because Alice may have changed her private or public key during this time; she may also claim that the file containing the signature is not authentic.
- One solution is a trusted third party.

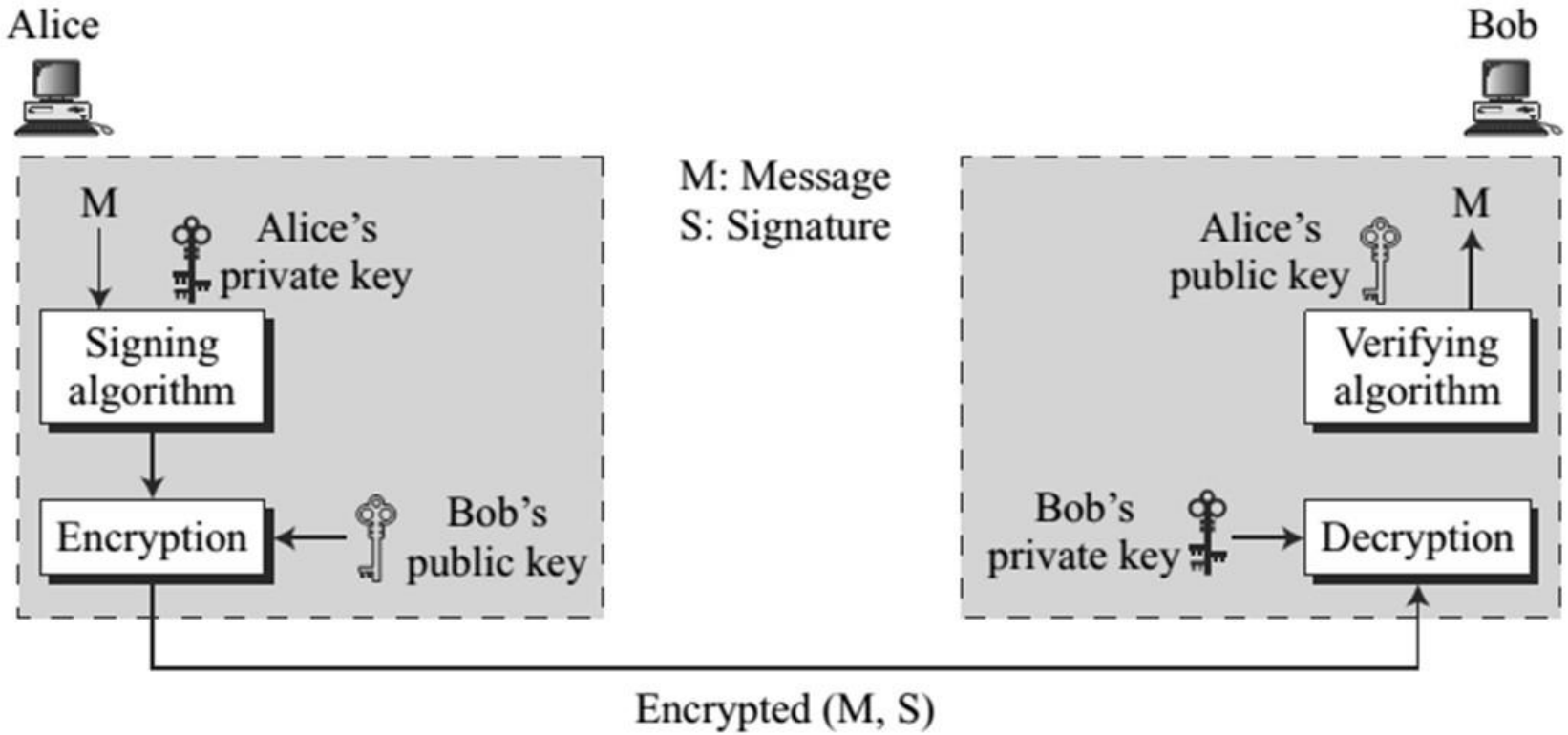
Digital Signature : Nonrepudiation



Digital Signature : Nonrepudiation

- Alice creates a signature from her message (S_A) and sends the message, her identity, Bob's identity, and the signature to the center.
- The center, after checking that Alice's public key is valid, verifies through Alice's public key that the message came from Alice.
- The center then saves a copy of the message with the sender identity, recipient identity, and a timestamp in its archive.
- The center uses its private key to create another signature (S_T) from the message.
- The center then sends the message, the new signature, Alice's identity, and Bob's identity to Bob.
- Bob verifies the message using the public key of the trusted center.
- **Nonrepudiation can be provided using a trusted party.**

Digital Signature : Confidentiality



A digital signature does not provide confidential communication.

If confidentiality is required, the message and the signature must be encrypted using either a secret-key or public-key cryptosystem.

A digital signature does not provide privacy.
If there is a need for privacy, another layer of encryption/ decryption must be applied.

ATTACKS ON DIGITAL SIGNATURE

- Key-Only Attack
  - In the key-only attack, Eve has access only to the public information released by Alice.
  - To forge a message, Eve needs to create Alice's signature to convince Bob that the message is coming from Alice.
  - This is the same as the ciphertext-only attack we discussed for encipherment.
- Known-Message Attack
  - In the known-message attack, Eve has access to one or more message-signature pairs.
  - In other words, she has access to some documents previously signed by Alice.
  - Eve tries to create another message and forge Alice's signature on it.
  - This is similar to the known-plaintext attack we discussed for encipherment.

ATTACKS ON DIGITAL SIGNATURE

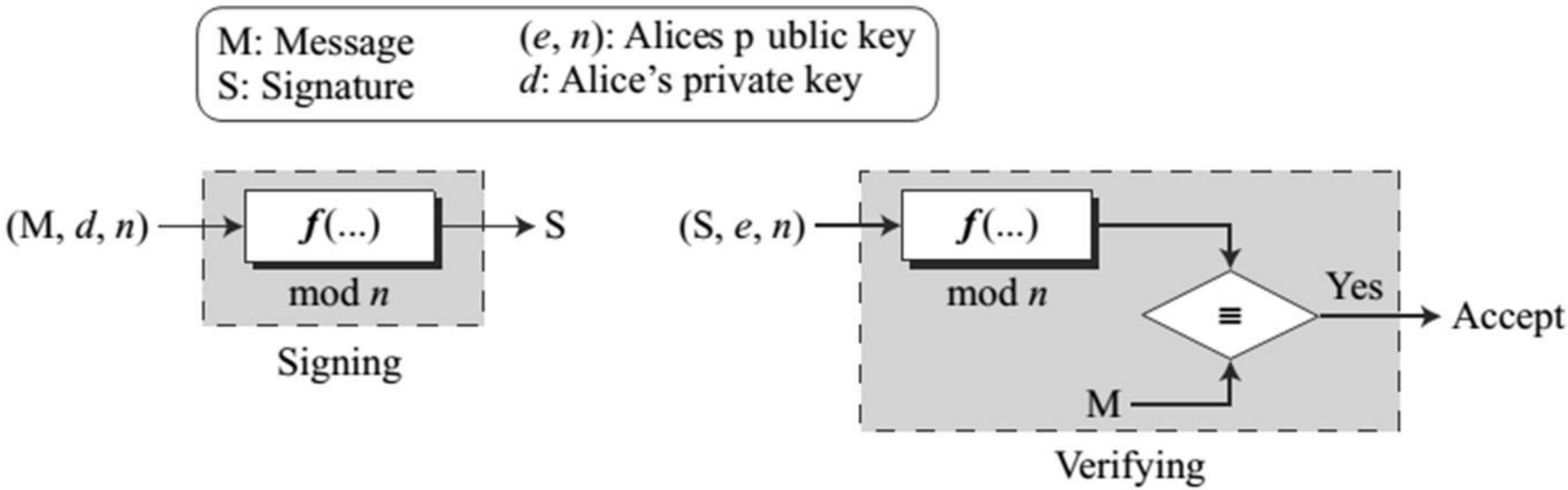
- Chosen-Message Attack
  - In the chosen-message attack, Eve somehow makes Alice sign one or more messages for her.
  - Eve now has a chosen-message/signature pair.
  - Eve later creates another message, with the content she wants, and forges Alice's signature on it.
  - This is similar to the chosen-plaintext attack we discussed for encipherment.
- If the attack is successful, the result is a forgery.
- We can have two types of forgery:
  - existential and
  - selective.

Forgery Types

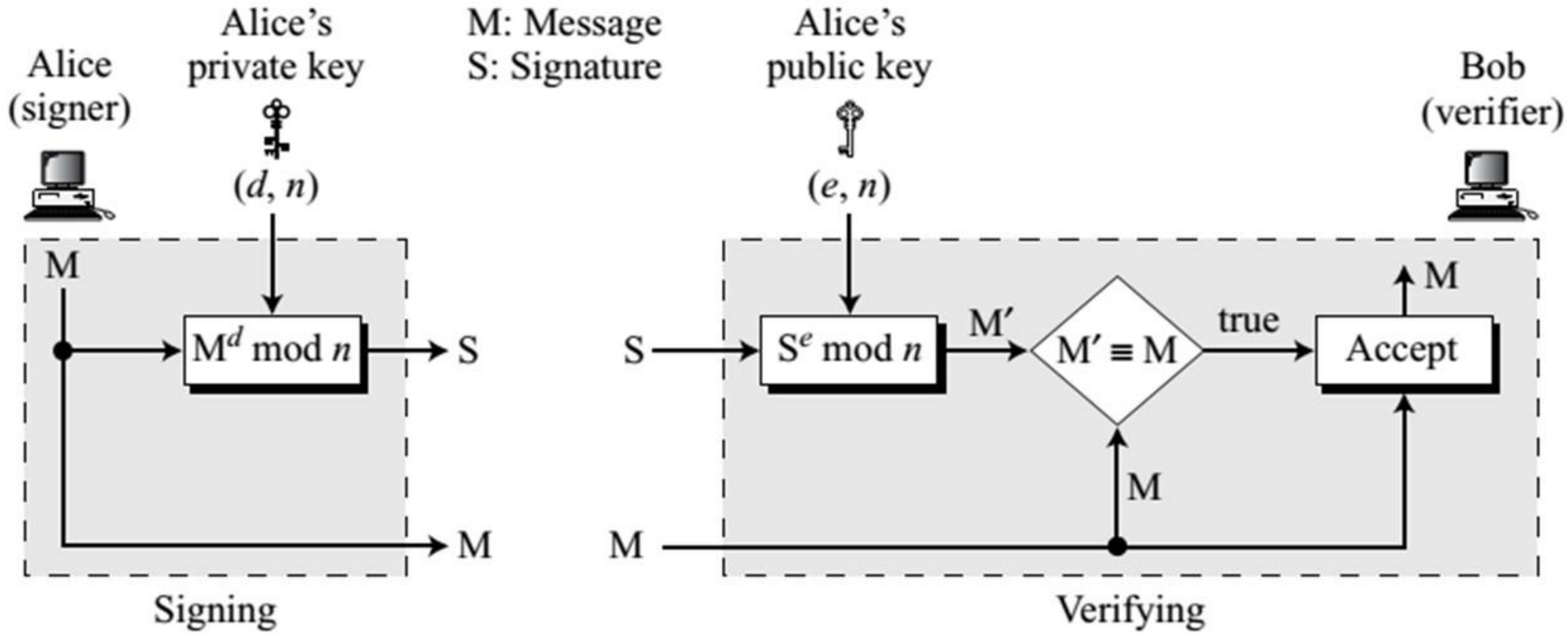
- Existential Forgery
  - In an existential forgery, Eve may be able to create a valid message-signature pair, but not one that she can really use.
  - In other words, a document has been forged, but the content is randomly calculated.
  - This type of forgery is probable, but fortunately Eve cannot benefit from it very much.
  - Her message could be syntactically or semantically unintelligible.
- Selective Forgery
  - In selective forgery, Eve may be able to forge Alice's signature on a message with the content selectively chosen by Eve.
  - Although this is beneficial to Eve, and may be very detrimental to Alice, the probability of such forgery is low, but not negligible.

DIGITAL SIGNATURE SCHEMES

RSA Digital Signature Scheme



RSA Digital Signature Scheme



$$M' \equiv M \pmod{n} \quad \rightarrow \quad S^e \equiv M \pmod{n} \quad \rightarrow \quad M^{d \times e} \equiv M \pmod{n}$$

RSA Digital Signature Scheme

- For the security of the signature, the value of p and q must be very large.
- As a trivial example, suppose that Alice chooses p = 823 and q = 953, and calculates n = 784319.
- The value of $\phi(n)$ is 782544. Now she chooses e = 313 and calculates d = 160009.
- At this point key generation is complete. Now imagine that Alice wants to send a message with the value of M = 19070 to Bob.
- She uses her private exponent, 160009, to sign the message

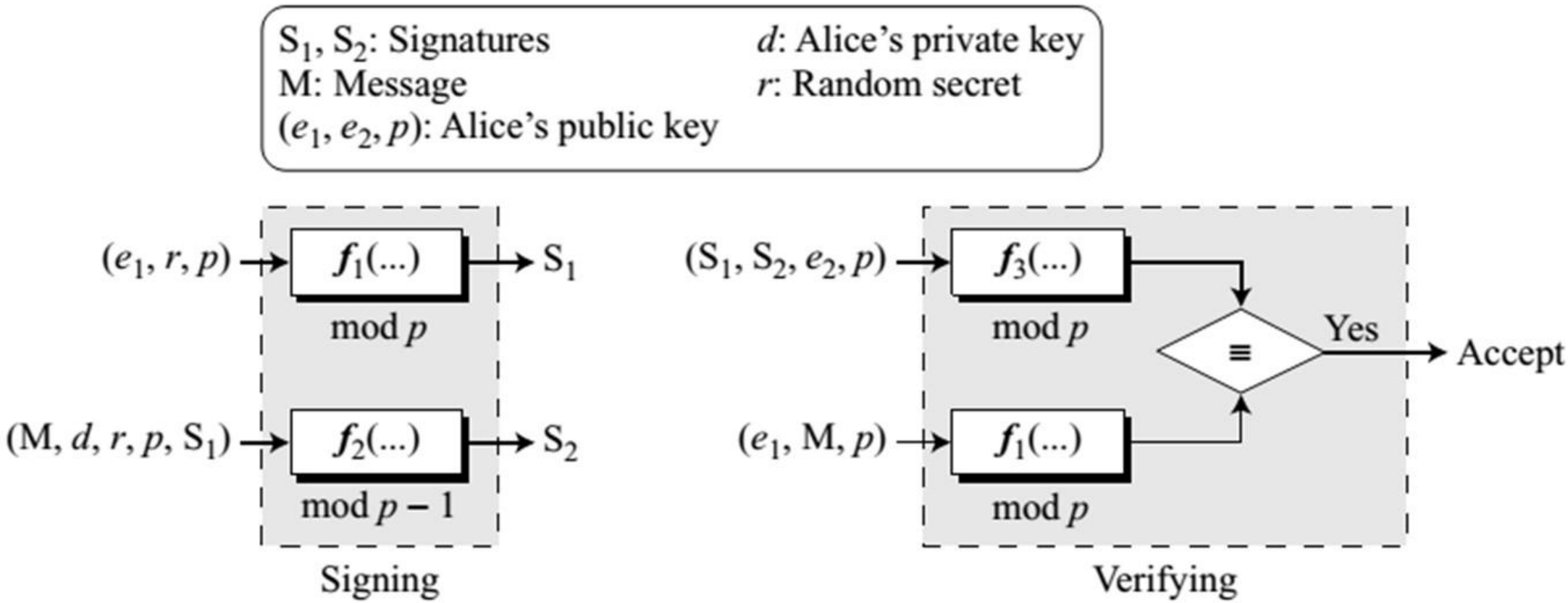
$$\text{M: } 19070 \quad \rightarrow \quad \text{S} = (19070^{160009}) \bmod 784319 = 210625 \bmod 784319$$

- Alice sends the message and the signature to Bob. Bob receives the message and the signature. He calculates

$$\text{M}' = 210625^{313} \bmod 784319 = 19070 \bmod 784319 \quad \rightarrow \quad \text{M} \equiv \text{M}' \bmod n$$

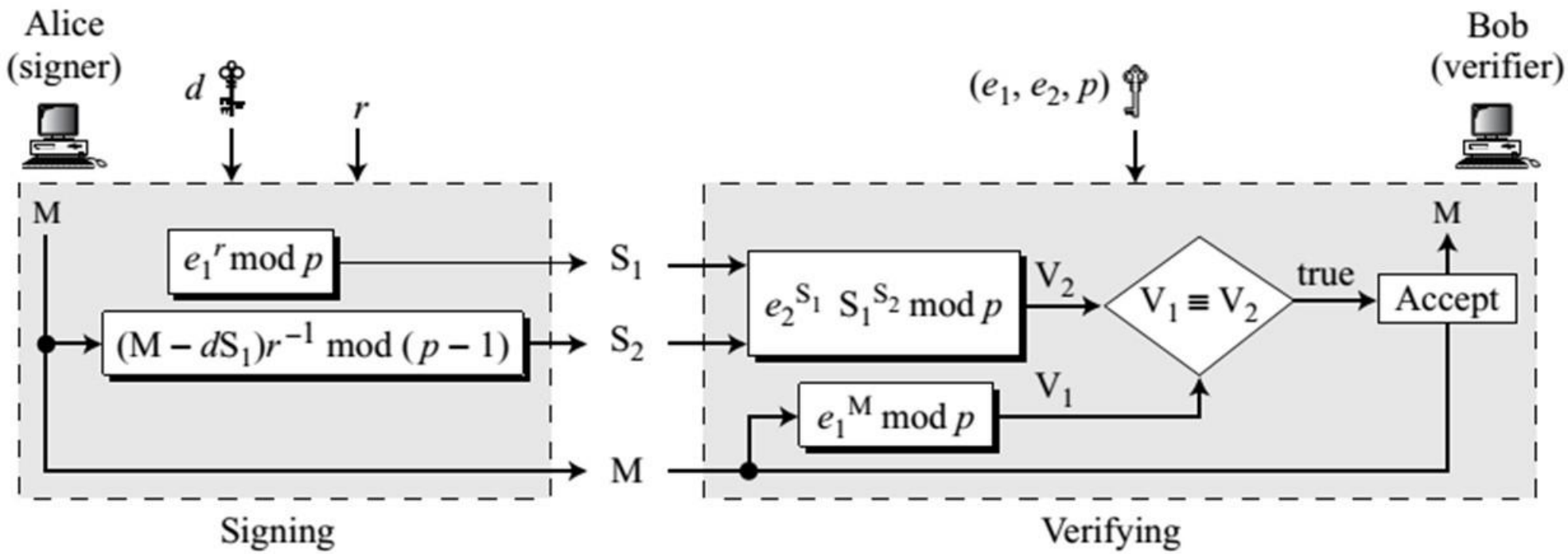
- Bob accepts the message because he has verified Alice's signature.

ElGamal Digital Signature Scheme



ElGamal Digital Signature Scheme

M: Message
S_1, S_2: Signatures
V_1, V_2: Verifications
r: Random secret
d: Alice's private key
(e_1, e_2, p): Alice's public key



ElGamal Digital Signature Scheme

$$V_1 \equiv V_2 \pmod{p} \rightarrow e_1^{M} \equiv e_2^{S_1} \times S_1^{S_2} \pmod{p} \equiv (e_1^{d})^{S_1} (e_1^{r})^{S_2} \pmod{p} \equiv e_1^{d S_1 + r S_2} \pmod{p}$$

We get: $e_1^{M} \equiv e_1^{d S_1 + r S_2} \pmod{p}$

ElGamal Digital Signature Scheme

- Here is a trivial example. Alice chooses p = 3119, e1 = 2, d = 127 and calculates e2 = 2^{127} mod 3119 = 1702. She also chooses r to be 307. She announces e1, e2, and p publicly; she keeps d secret. The following shows how Alice can sign a message.

$$M = 320$$

$$S_1 = e_1{}^r = 2^{307} = 2083 \bmod 3119$$

$$S_2 = (M - d \times S_1) \times r^{-1} = (320 - 127 \times 2083) \times 307^{-1} = 2105 \bmod 3118$$

Alice sends M, S_1, and S_2 to Bob. Bob uses the public key to calculate V_1 and V_2.

$$V_1 = e_1{}^M = 2^{320} = 3006 \bmod 3119$$

$$V_2 = d^{S_1} \times S_1{}^{S_2} = 1702^{2083} \times 2083^{2105} = 3006 \bmod 3119$$

Because V_1 and V_2 are congruent, Bob accepts the message and he assumes that the message has been signed by Alice because no one else has Alice's private key, d.

ElGamal Digital Signature Scheme

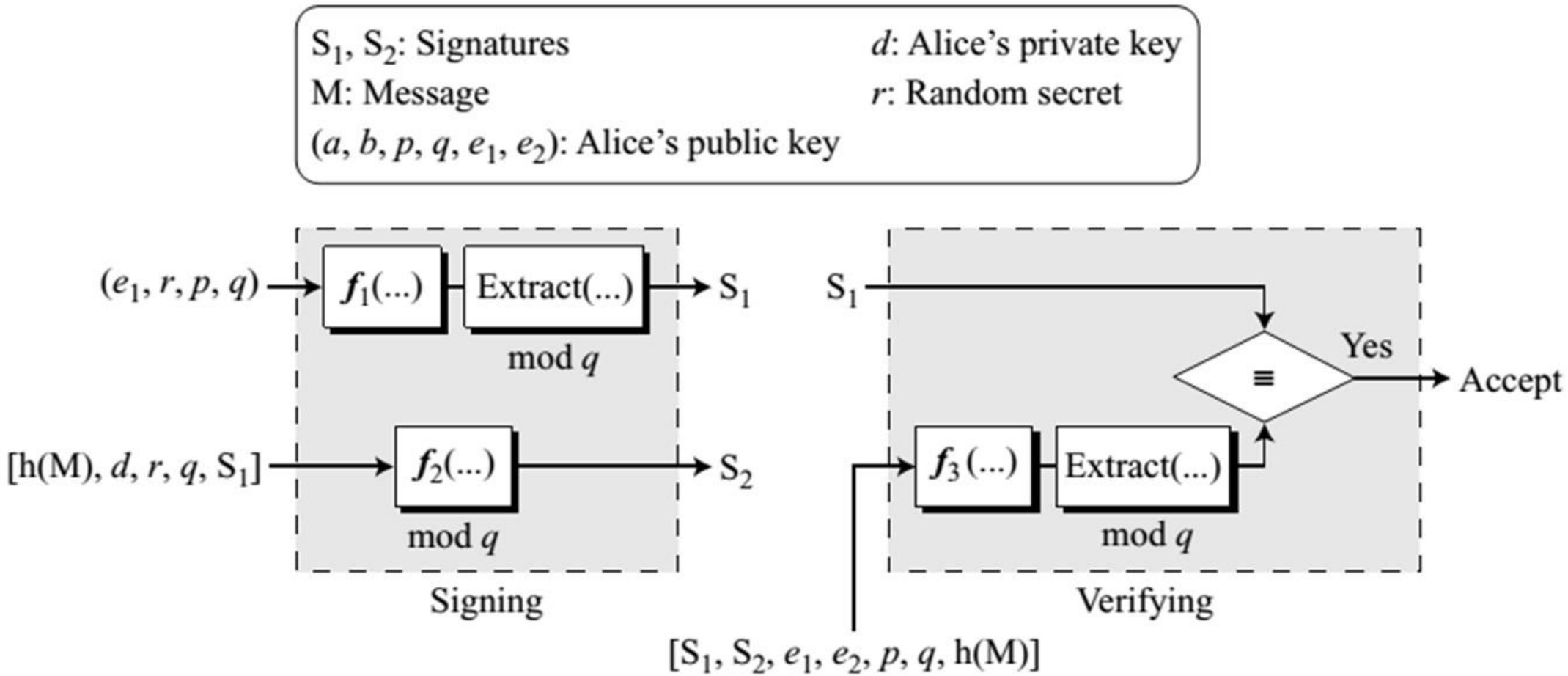
Now imagine that Alice wants to send another message, M = 3000, to Ted. She chooses a new r, 107. Alice sends M, S_1, and S_2 to Ted. Ted uses the public keys to calculate V_1 and V_2.

$$M = 3000$$
$$S_1 = e_1{}^r = 2^{107} = 2732 \bmod 3119$$
$$S_2 = (M - d \times S_1)\, r^{-1} = (3000 - 127 \times 2083) \times 107^{-1} = 2526 \bmod 3118$$

$$V_1 = e_1{}^M = 2^{3000} = 704 \bmod 3119$$
$$V_2 = d^{S_1} \times S_1{}^S = 1702^{2732} \times 2083^{2526} = 704 \bmod 3119$$

Because V_1 and V_2 are congruent, Ted accepts the message; he assumes that the message has been signed by Alice because no one else has Alice's private key, d. Note that any person can receive the message. The goal is not to hide the message, but to prove that it is sent by Alice.

Elliptic Curve Digital Signature Scheme

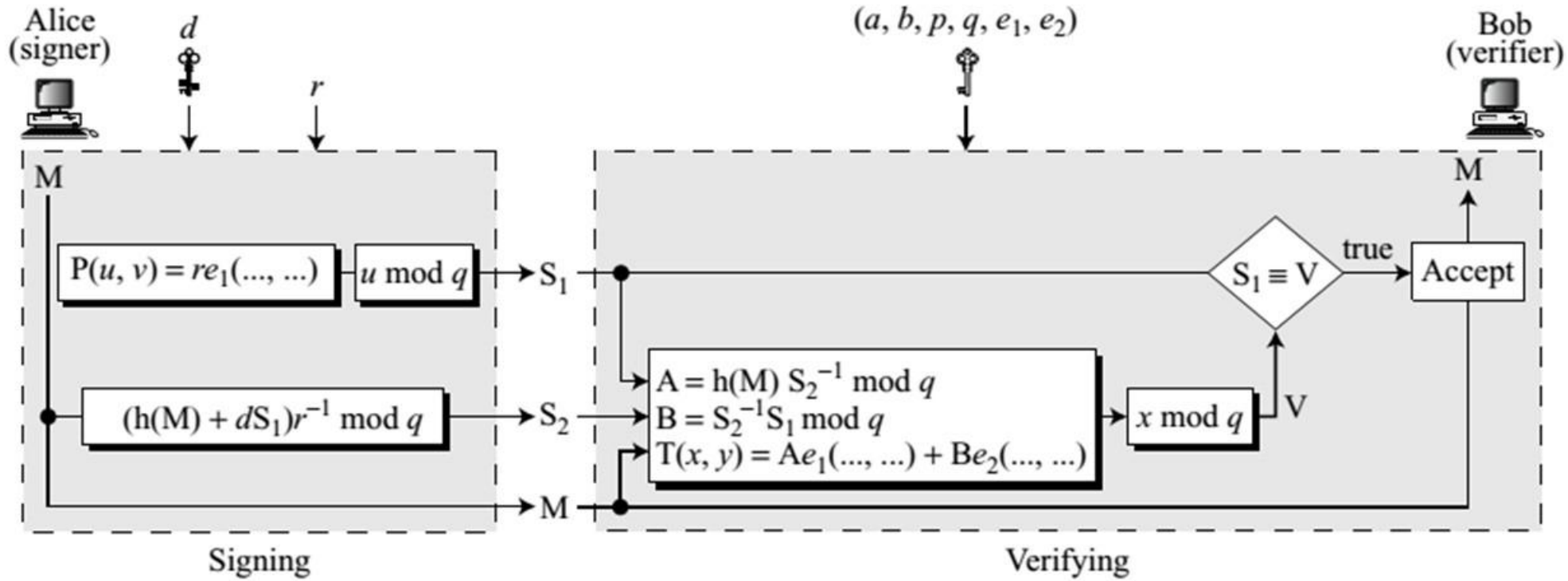


Elliptic Curve Digital Signature Scheme

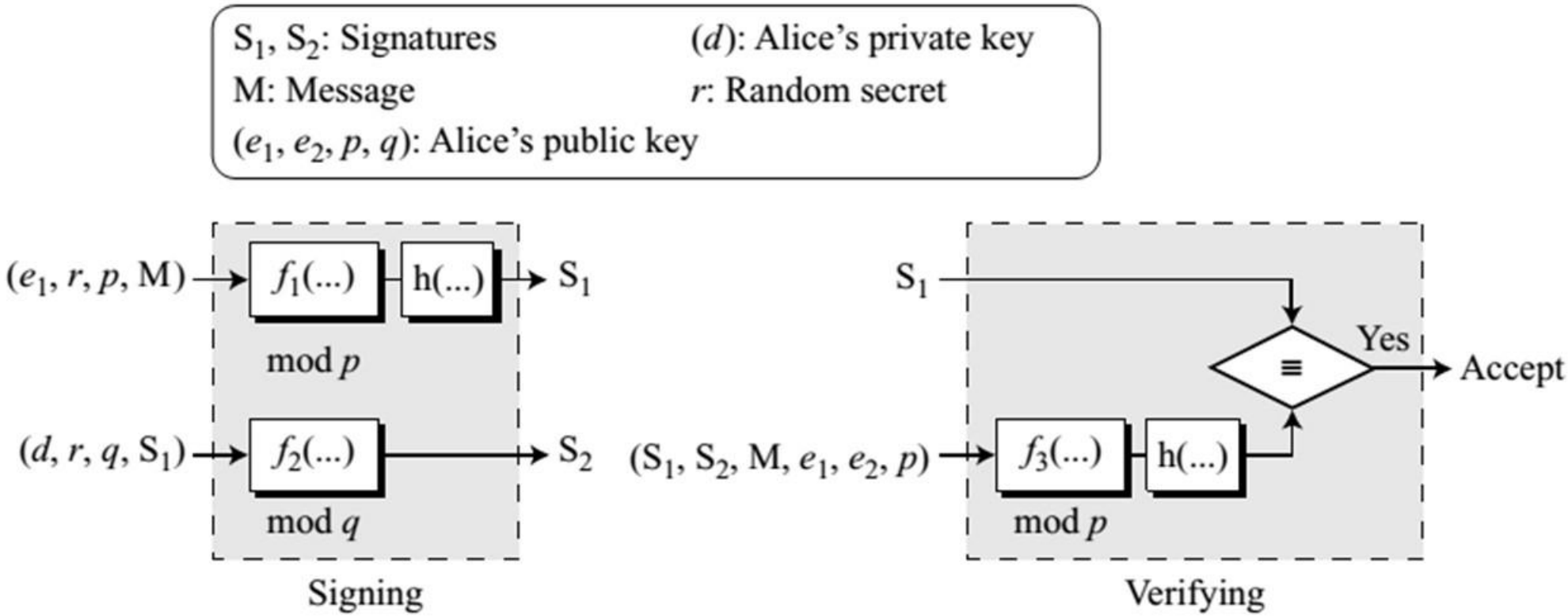
M: Message
S_1, S_2: Signatures
V: Verification

r: Random secret
d: Alice's private key
(a, b, p, q, e_1, e_2): Alice's public key

P(u, v), T(x, y): Points on the curve
h(M): Message digest
A, B: Intermediate results



Schnorr Digital Signature Scheme



Schnorr Digital Signature Scheme

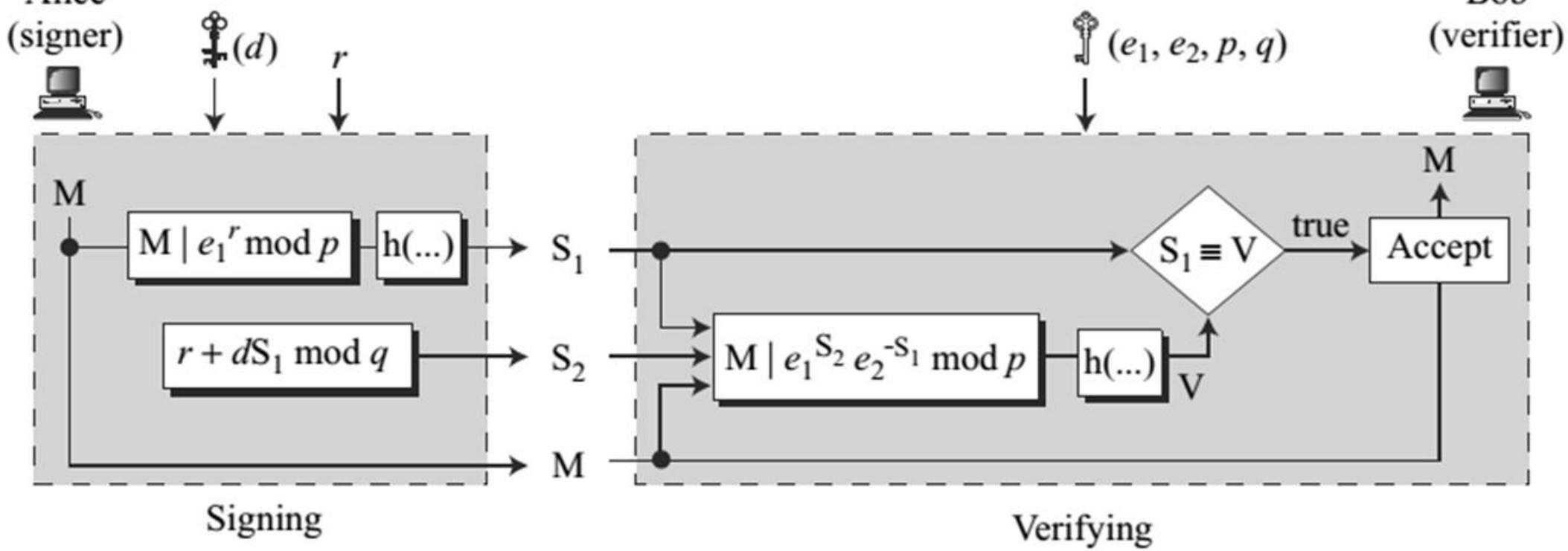
Key Generation

Before signing a message, Alice needs to generate keys and announce the public ones to the public.

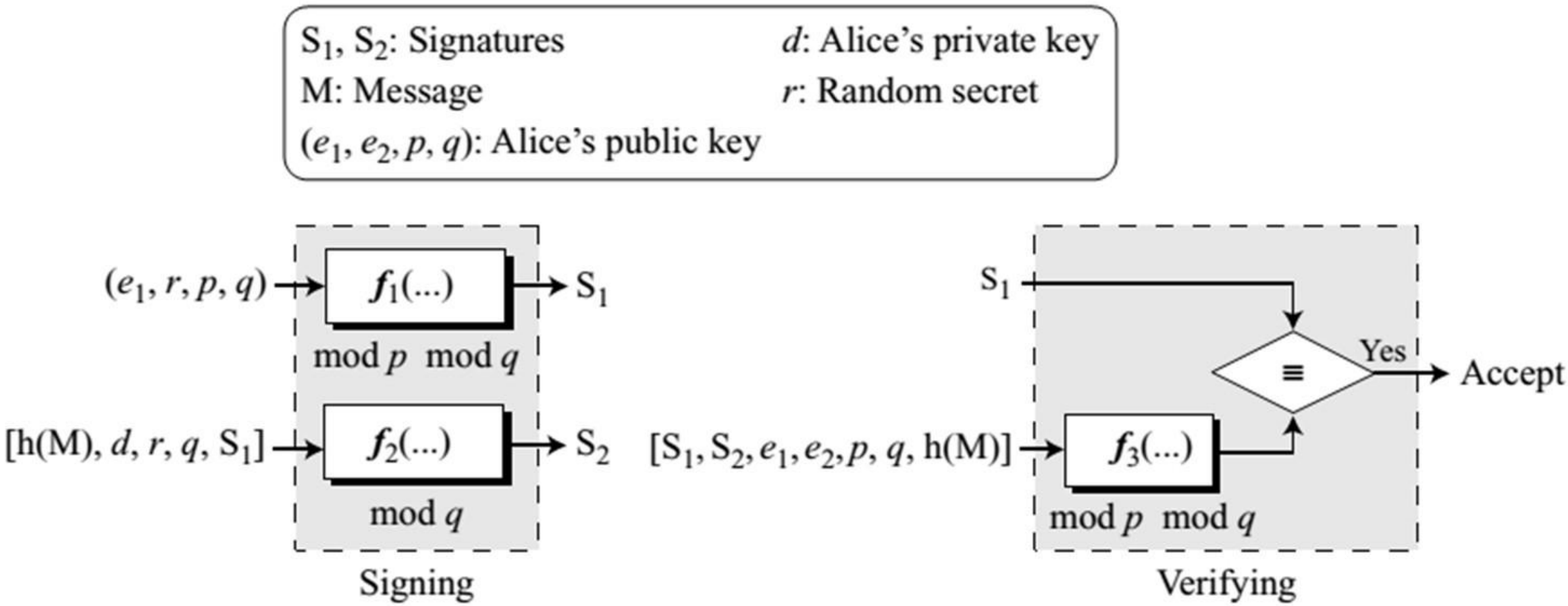
1. Alice selects a prime p, which is usually 1024 bits in length.
2. Alice selects another prime q, which is the same size as the digest created by the cryptographic hash function (currently 160 bits, but it many change in the future). The prime q needs to divide $(p - 1)$. In other words, $(p - 1) = 0 \bmod q$.
3. Alice chooses e_1 to be the qth root of 1 modulo p. To do so, Alice chooses a primitive element in $\mathbf{Z}_p$, e_0 (see Appendix J), and calculates $e_1 = e_0^{(p-1)/q} \bmod p$.
4. Alice chooses an integer, d, as her private key.
5. Alice calculates $e_2 = e_1^{d} \bmod p$.
6. Alice's public key is (e_1, e_2, p, q); her private key is (d);

Schnorr Digital Signature Scheme

M: Message
S_1, S_2: Signatures
V: Verification
r: Random secret
(d): Alices private key
(e_1, e_2, p, q): Alice's public key
|: Concatenation
h(...): Hash algorithm

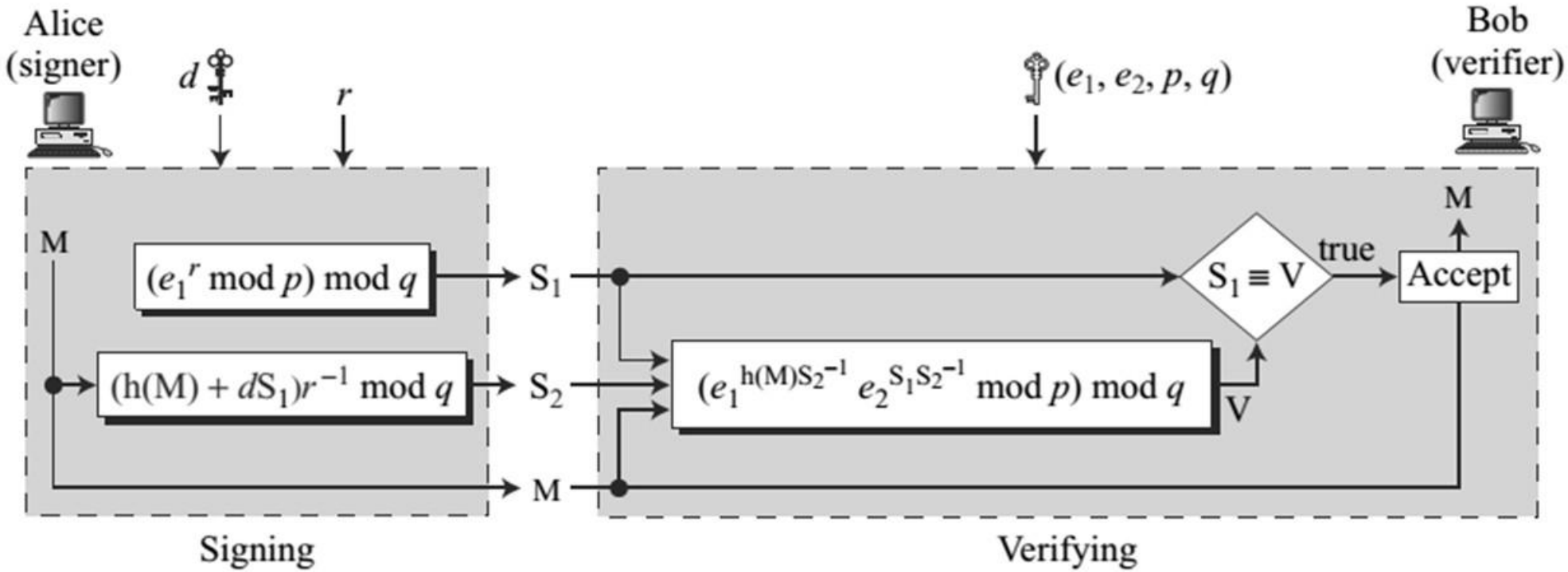


Digital Signature Standard (DSS)



Digital Signature Standard (DSS)

M: Message
S_1, S_2: Signatures
V: Verification
r: Random secret
d: Alices private key
(e_1, e_2, p, q): Alice's public key
h(M): Message digest



Digital Signature Standard (DSS)

Key Generation

Before signing a message to any entity, Alice needs to generate keys and announce the public ones to the public.

1. Alice chooses a prime p, between 512 and 1024 bits in length. The number of bits in p must be a multiple of 64.
2. Alice chooses a 160-bit prime q in such a way that q divides $(p - 1)$.
3. Alice uses two multiplication groups $<\mathbf{Z}_p^*, \times>$ and $<\mathbf{Z}_q^*, \times>$; the second is a sub-group of the first.
4. Alice creates e_1 to be the qth root of 1 modulo p ($e_1^p = 1 \bmod p$). To do so, Alice chooses a primitive element in $\mathbf{Z}_p$, e_0, and calculates $e_1 = e_0^{(p-1)/q} \bmod p$.
5. Alice chooses d as the private key and calculates $e_2 = e_1^d \bmod p$.
6. Alice's public key is (e_1, e_2, p, q); her private key is (d).

Tutorial

- Explore the security attacks on schemes